

Construction Materials

It is important to determine early on a starting point for the materials that may be used to construct the robot. The goal of this league will be to minimize the cost of build robots so decisions on what items can be used will be critical to keeping costs down. In some ways I would like to emulate BEST Robotics, that strictly limits materials to the kit. One key difference being that the participants will need to purchase all the materials used to create their robot. At the end of the competition the players will be able to keep their robot as it has been built with components they purchased. Because participants will be purchasing everything (as in the FIRST Robotics Competition) I would like to keep costs down by proposing stricter limits on the items that can be used (similar to BEST). Also, the robots will need to be significantly smaller and less powerful (likely powered by AA batteries or 9-volt).

Electronics

Electronics are any components that perform computation or are powered using electricity. All electronic components must be available in the kit. Electronics do not have to be purchased in the kit, but the number of components used by the robot cannot exceed the number of that component in the kit.

Stock material

Stock materials are raw materials that can be processed into components by the purchaser of the materials. Stock materials do not include prefabricated components. The material must be sold with the intent of further processing and must be processed by the robot builder in some way that adds value for the competition. Acceptable stock material includes:

Wood

3D printing filament

2-part resin, epoxy and silicone

Cardboard

PVC

Fasteners and Adhesive

All fasteners and adhesives can be used as long as they are used as fasteners and adhesives.

Prefabricated Parts

Prefabricated parts are single or multi-material components that are manufactured and not intended to be further processed before being used in a project. In order to keep costs down, the use of prefabricated parts will be limited to the listed parts.

Specific Wheels

Caster Wheels

Brackets

Hinges

Magnets

Wire

Springs

Cables

Ball Bearings

Rope or Twine

Rubber Bands

Robot Size and Weight

The robot will need to fit into a much smaller size than robots in competitions such as BEST and FIRST. To keep the size small enough to be able to print all components on inexpensive 3D printers, I propose limiting the size of the robot to fit within an 8"x8"x8" cube. I would also propose keeping the weight under 8lbs to limit the size of the battery required to move the robot. These could be changed later as we start to play around with the materials.

The Kit

The following items can potentially be included in the kit. What is included will affect the final cost of the kit. It needs to be determined if some prefabricated parts need to be included, or if only electronic components should be included to keep kit costs down.

Electronic Components

Metal Gear Servo - \$5

Game Controller - \$15 to \$20

ESP32 + Breakout Board - \$7

L298N Motor Controller - \$3

Cheap Yellow Motor (2 count) - \$3

Buck Converters (2 count) - \$5

Prefabricated Parts

12v wire - \$1

6 AA Battery Pack - \$2

6 NiMH AA Battery - \$8

2ft Red Wire + 2ft Black Wire - \$2

24 M to F Jumper Wires - \$1

100 Spacers - \$2

Yellow motor wheel (2 count) - \$4

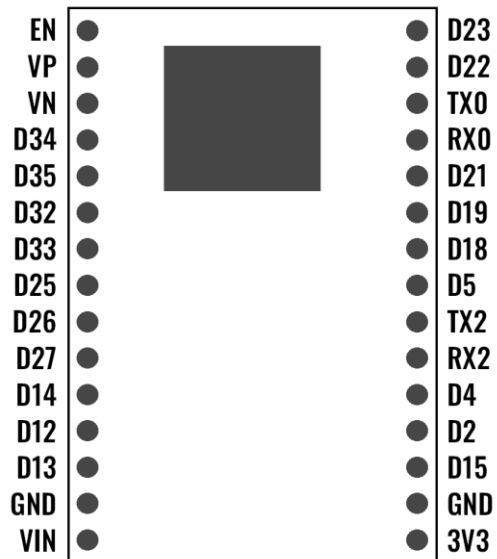
M3 x 40mm (10 count) - \$2

M3 Nuts (10 count) - \$1

M3 Wing Nut (5 count) - \$2

2" Caster Wheel - \$4

ESP32



PWM Pins

D13, D14, D18, D19, D21-D23, D25-D27, D32, D33

Digital Output Pins (High / Low)

D5 (may have boot issues)

D13, D14, D18, D19, D21, D32, D33

Digital Read Only Pins

D34, D35, D36, D39

GPIO Pins to Avoid

6-11 (These do not have D designations)

Pins 6-11 are used for Flash Memory access, and trying to put a current on one of them could corrupt the system's Flash Memory.

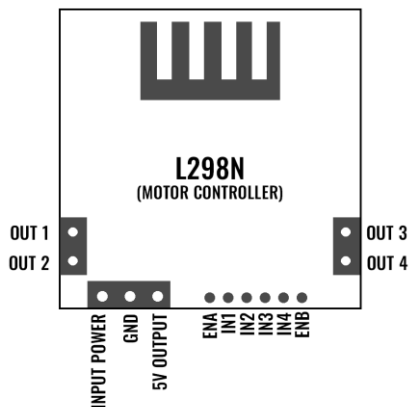
Pins used in Bootup

D2, D5, D12 and D15

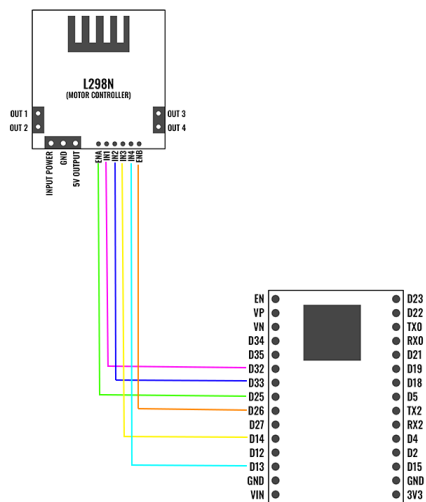
The ESP32 uses 3.3v for logic circuitry but has a VIN with a voltage regulator for 5 volt (and sometimes greater) input to keep backwards compatibility with devices like Arduino.

TX0, RX0 and TX2, RX2 are serial transmission and receiver pins that are used for things like USB input.

L298N Motor Controller



Wiring the motor controller



Setting up the pins

In the Arduino `setup` function, you need to call the `pinMode` function to set the pin mode to `OUTPUT`. If you don't do this, you will be unable to write to the pin. You can do a `digitalWrite` to a pin that will either put a current on the pin `HIGH` or make sure there is no current on the pin `LOW`. The IN1, IN2, IN3 and IN4 pins on the ESP32 are digital pins. IN1 and IN2 are used for the motor connected to OUT 1 and OUT 2. If IN1 is HIGH and IN2 is LOW, the motor connected to OUT 1 and OUT 2 is put into forward, and if IN1 is LOW and IN2 is HIGH, that motor is in reverse. If both pins are HIGH the brakes are applied to that motor (energy is used to hold the motor's position). If both pins are LOW, no power is applied to the motor. The pins IN3 and IN4 work the same way for the motor connected to OUT 3 and OUT4. If the currents on pins IN1 and IN2 are not both LOW, the PWM current on ENA determines what percentage of the current from the main input source will be used to drive the motor.

PWM

PWM stands for "Pulse Width Modulation." It is one way that a larger set of numbers (not just 1 or 0) can be sent over a single data pin. Let's say we want to run one of our motors at 50% of its full speed by sending a PWM signal to the ENA pin. Because the default resolution on the PWM signal is 8-bit, you would do the following `analogWrite` to your ENA pin:

```
analogWrite(ENA_PIN, 128);
```

The signal on that pin would be HIGH for half the time and LOW for half the time. This would be received as an 8-bit value of 128 by the motor controller.

PWM of 128 (on an 8-bit channel) would look like this: HIGH _____ | _____ LOW

Note on two L298N Motor Controllers:

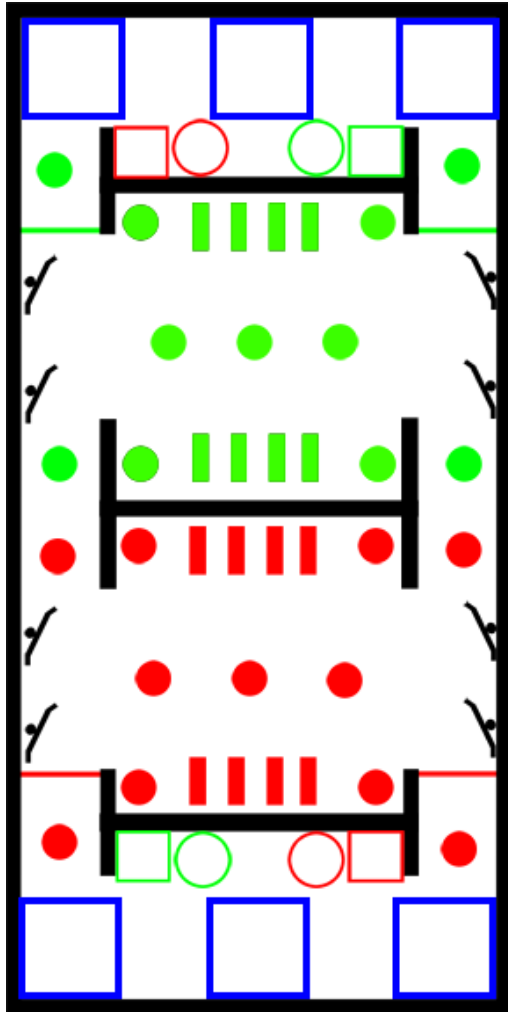
If we want to expand to two L298N motor controllers we will need to add a shift register.

The L298N requires 3 pins / motor, adding two additional motors will require six additional output pins. Each servo only requires a single PWM output pin from the ESP32.

The Game

The game will consist of a team of robots (2 or 3) that attempt to pick up and move items back to a goal location in their home zone. Another possible task will be flipping switches to give additional points to their team. Items colored the same color as their home zone

will be worth less points in goal than items with the colors of the opposing team's home zone. Switches will start with half of the switches on each team's home zone color. Points will be awarded if one team has more switches on their color than the opposing team. No player can enter the opposing team's home zone.



Robots are not allowed to intentionally collide with each other. Robots can be penalized by picking up the offending robot and moving it back to the home zone, possibly with an additional penalty time depending on the infraction. If a robot stops functioning during play, a player may request that the ref bring the robot back to the home zone for the player to repair. A player may take their robot out of the home zone in order to repair it, and then may redeploy it to their designated position in the home zone.